Production of seasonally adjusted series

Anna Smyk Tanguy Barthelemy

2026-04-02

Table of contents

JDemetra+ and R Cookbook

Seasonal adjustment is a key process in official statistics, as it is necessary for the dissemination and interpretation of a large number of economic indicators. In an effort to harmonize practices, Eurostat has recommended since 2015 the use of the open source software JDemetra+, which provides access to the two most popular seasonal adjustment algorithms in European statistical agencies: X-13 Arima and Tramo-Seats, via a graphical user interface and a family of R packages.

These pages describe and illustrate how to take advantage of JDemetra+ version 3 to build efficient production processes that can adjust several thousand series with individualized settings, while controlling statistical quality and minimizing manual fine-tuning.

Introduction

Seasonal adjustment is a key process in official statistics. The dissemination of most economic indicators requires that they first be purged of quasi-periodic, seasonal, and calendar movements in order to facilitate the interpretation of short-term developments and the identification of turning points and medium- and long-term trends (Bell and Hillmer 1984). However, implementing a standardized and flexible process for producing seasonally and calendar adjusted (sa) series that complies with European statistical system best practices (Eurostat 2024) is costly. In addition to producing seasonally adjusted series, it must also provide integrated quality indicators (Kirchner and alli 2018) and allow for manual adjustment by experts, often under significant time constraints. At INSEE, all seasonal adjustment processes are based on JDemetra+ (Smyk and alli 2026), officially recommended by Eurostat since 2015. This open source software for time series analysis provides access to the two most popular seasonal adjustment algorithms in European official statistics: X-13 Arima (US-Census-Bureau 2015) and Tramo-Seats (Gomez and Maravall 1996). The algorithms are written in Java and accessible via a Graphical User Interface and a family of R packages: the rjdverse .

Version 3 of JDemetra+, available since 2023, offers new R tools to improve production processes. In addition, the Covid crisis has highlighted the need for fine-grained parameterization on a very large scale, such as assigning specific outlier profiles to each series for several hundred series. Finally, the emphasis in recent years on the transition from SAS to R at INSEE is promoting the acceptance of these new tools in a general context of modernization. Therefore, the current context is very favorable for reviewing and optimizing existing processes.

Each year, INSEE's Statistical Methods Department undertakes a number of process redesigns, disseminates best practices, and trains producers in the use of the JDemetra+ software. However, in order to clarify the construction and optimization of seasonality adjustment processes, an example of a method and a presentation of the latest tools are necessary.

Currently, producers of seasonally adjusted series have access to theoretical information on seasonal adjustment algorithms, such as the Handbook on Seasonal Adjustment , a guide to best practices , the JDemetra+ software documentation , and possibly notes reflecting the historical sedimentation of practices in

your institution. One piece seems to be missing from this puzzle: a specific example of implementation using the most up-to-date tools. The rest of this article aims to describe and illustrate how to take advantage of JDemetra+ version 3.x to build efficient production processes, adjust up to several thousand series with individualized settings, control statistical quality, and target manual fine-tuning.

The production of seasonally adjusted series generally involves three stages: the process of installation, during which methodological changes are most significant; annual revision campaigns; and infra-annual revision campaigns. Production processes can be based on *workspaces*—a data structure specific to JDemetra+ that allows for the use of a graphical interface—or be built entirely in R.

Annual campaigns require tools for comparing and prioritizing models in need of manual fine-tuning. The R package JDCruncheR can be used for this purpose. Infra-annual production requires data revision policies and tools for analysis of revisions to validate figures before publication. We detail these three phases by showing the advantages and disadvantages of different configurations, presenting available tools, and comparing execution times. The article includes R code snippets, and a complete example is available in the {rjd3production} package .

Following a brief overview of the statistical operations comprising the seasonal adjustment process and the JDemetra+ software, the second section outlines the construction of a production process. The third section covers annual and infra-annual reviews of parameters. The appendices provide a more detailed presentation of the reviewed tools.

Part 1: Seasonal Adjustment and Production

The statistical process

Seasonal adjustment removes periodic movements from the data series that mask cyclical variations and long-term trends. For this reason, most economic indicators are published seasonally (and calendar) adjusted (SA). Seasonal adjustment requires estimating a seasonal component that captures all periodic intra-annual movements in order to remove it from the raw series $Y_{\text{CVS}} = Y - S$. To do this, the raw series is broken down into three unobservable components: seasonality, trend, and irregularity ($Y = S + T + I$), where the trend T represents long-term movements and I represents noise generated by exceptional events (strikes, weather conditions, short-term shocks, etc.) and all measurement errors. The algorithms most commonly used today by European national statistical institutes and central banks are Tramo-Seats (Gomez and Maravall 1996) and X13-Arima (US-Census-Bureau 2015). These methods proceed in two stages: the raw series is not directly decomposed but first linearized, by (permanently) removing calendar effects and (temporarily) removing outliers using Reg-Arima modelling.

Objectives

The main objective is, of course, to produce seasonally adjusted (SA) series, particularly with a view to publication. However, to ensure the quality of the seasonal adjustment, a broader output is necessary. The process must therefore allow the diagnostics to be viewed (seasonality tests, residual seasonality tests, and residual calendar effects for the seasonally adjusted series, etc.) and the parameters to be adjusted in light of the problems identified by these diagnostics. Intermediate series, in addition to the final SA series, such as unobservable components, the linearized series, and seasonal factors (including forecast factors), are often useful.

Steps

Seasonal adjustment begins with a seasonality test to ensure that this operation is necessary for the series in question. If this is not the case, the raw series will also be published as SA. In practice, this test is often performed at the end of the seasonal adjustment process, which allows it to be performed on the linearized series (from which the S factors will be extracted) and to always keep the same input data set, excluding only the series that are not to be seasonally adjusted at the dissemination stage.

A correction for calendar effects must often be implemented or at least tested [1]. The first estimation (pre-adjustment and decomposition) can then be made, probably with a largely automatic specification, and a quality report can be generated to improve the quality of the seasonal adjustment. We refer here to a quality report and not simply to diagnostics, as the former consists of formatting the diagnostics and summarizing them in a score that allows a *selective editing* approach to prioritize manual fine-tuning in the series. It is generally necessary to quantify the revisions and the numerical impact of the choice of parameters, which will influence the choice of final parameters. Once these elements have been reviewed, a second estimation can be made, and the process may be repeated until a final set of parameters is reached.

Table 1: The steps

Domain	Operations
Correction of calendar effects	Creation of a reference calendar
	Generation of regression sets
	Selection of the best set for each series
Customization of specifications	Assignment of calendar regressors
	Assigning outliers
	Other parameters
Estimation	Preliminary estimation
	Output generation
Quality Control	Quality Report
	Manual fine tuning
	Final estimation
	Final output generation

[1] Eurostat guidelines recommend to correct calendar effects that have ex-ante relevance.

The production process

Properties

Production inherently requires the automation of as many operations as possible (definition of specifications, estimation, production of a quality report, application of revision policies), but also, in the case of seasonality adjustment, the visualization of diagnostics and graphs in an environment that allows for rapid manual adjustment of parameters.

The process can be divided into three distinct phases based on their objectives and the type of intervention they require: implementation of a production process, annual campaigns, and infra-annual campaigns.

Set-up

The process must be initialized by making medium-term choices that will not, in principle, be called into question for a period of approximately five years. User requirements are then specified: the variables of interest to be seasonally adjusted are selected, as well as the level of the nomenclature and the direct (seasonal adjustment at each level) or indirect (aggregation of seasonally adjusted series) approach. See Guidelines chapter 5.4 for more details. The relevance of a calendar effect correction must be tested. This is also the time to determine the length of the estimation period (*Ibid.*, Chapter 8) and the depth of revisions during publication (*Ibid.*, Chapter 6). This is also when the algorithm and software to be used should be chosen.

Annual and infra-annual reviews

Approximately once a year, the producer updates the statistical parameters based on new raw data (*Ibid.*, Chapter 6). One method is to compare current models with an automatic re-estimation, as described in Part 3.

During infra-annual production, the goal is to minimize technical revisions. It is recommended that you review only a limited number of parameters, and you may even use the seasonal coefficients predicted during the last annual campaign. The focus should be on revisions and on the appearance or disappearance of outliers in the recent period.

The rest of this article presents a procedure and the tools needed to implement the three phases described above. As part of harmonizing statistical practices

and tools, Eurostat has recommended JDemetra+ to member countries of the European Statistical System for seasonal adjustment since 2015. We will therefore use this software. The following section briefly presents its main features relating to the production of seasonally adjusted series.

JDemetra+: a tool for seasonal adjustment

JDemetra+ is an open-source library of algorithms for time series analysis. These are accessible via a Graphical User Interface (GUI), which can be enhanced with plug-ins and an ecosystem of R packages, the rjdverse. In addition to seasonal adjustment, they also enable nowcasting, benchmarking, temporal disaggregation, trend estimation, and revision analysis. The development and promotion of JDemetra+ are co-financed by Eurostat through successive grants and currently within the COSA project.

Seasonal adjustment is the area for which JDemetra+ is best known. The software was initially developed as a simple graphical interface, based on the original programs for the two algorithms X12-arima and Tramo-Seats. The aim was to provide users with a user-friendly view of the results, diagnostics, and parameters, and to harmonize this for both algorithms. The software was initially called Demetra, the goddess of agriculture and the seasons. When the Fortran programs of the US Census Bureau and the Bank of Spain were rewritten in Java[2], Demetra became JDemetra+ (Grudkowska et al. 2013).

Algorithms

JDemetra+ now offers additional algorithms for seasonal adjustment, but only the two historical ones are really designed for mass data production, as they allow parameters to be stored in a *workspace* and use an automation module called Cruncher. In our examples, we will refer to X13-Arima, but all recommendations remain valid with Tramo-Seats.

Table 2: Seasonal adjustment algorithms

Algorithm	GUI	Packages R	Suitable for production
X13-ARIMA		{rjd3x13}	yes
Tramo-Seats		{rjd3tramoseats}	yes

[2]Same structure but potentially different estimation and optimization procedures that could give rise to numerically non-strictly equal results.

Algorithm	GUI	Packages R	Suitable for production
X12+		{rjd3x11plus}	not yet
STL		{rjd3stl}	not yet
Basic Structural Models		{rjd3sts}	not yet

All of the above algorithms can be used to seasonally adjust monthly, quarterly, bimonthly, four-monthly, and biannual data. They also have extensions for high-frequency (infra-monthly) data, as detailed in Webel and Smyk (2024).

Tools

Graphical User Interface (GUI)

JDemetra+ is available as a Graphical User Interface designed to facilitate manual fine-tuning through an organized presentation of results and parameters. Operations are mainly performed by clicking buttons, which requires auxiliary automation tools. Using the GUI and the *Cruncher* also requires organizing data into a specific structure: the *workspace*.

Cruncher

The *Cruncher* avoids opening the interface and clicking when launching or refreshing an estimation. It is a command line executable module that we will use later with two R packages, rjwsacruncher for estimations and exporting results, and JDCruncher for formatting diagnostics. Another advantage of the *Cruncher* is its speed: it can be run on hundreds or thousands of series and produce results in a reasonable amount of time. The installation procedure and main functions are described in the documentation.

R packages

A family of packages, rjdverse provides direct access to JDemetra+ algorithms in R and automates operations related to the use of *workspaces*. The rjd3production package offers macro functions corresponding to complete stages in the production process.

Table 3: R packages for the production of seasonally adjusted series

Package	Useful functions for production
{rjd3toolkit}	Generation of calendar regressors, Customization of specifications, Tests
{rjd3x13}	Seasonal adjustment (Reg-Arima pre-adjustment and X11 decomposition)
{rjd3tramoseats}	Seasonal adjustment (TRAMO pre-adjustment and SEATS decomposition)
{rjwsacruncher}	*workspace* update and output export
{JDCruncheR}	Create a quality report for the *workspace* based on the outputs
{rjd3production}	Wrapper with macro functions for each production step
{rjd3workspace}	*workspace* creation and modification
{rjd3providers}	Input data management

The installation procedures for all those tools are to be found in the documentation.

Data structures

In the following paragraphs, we briefly describe the data structures and concepts specific to JDemetra+, which will be useful in the following sections.

Input series

The JDemetra+ interface accepts different types of data formats that can be imported from the menus in the providers tab.

- CSV files (specifying the field separator, date format, decimal separator, etc.)
- Excel files (only *.xlsx in version 3.x)
- XML files
- SDMX data sources
- ODBC or JDBC databases

All series, whether seasonally adjusted data or auxiliary variables, are imported in the same way. For estimations, R packages work with TS class objects.

Outputs

JDemetra+ generates different types of outputs:

- Series (seasonally adjusted, seasonal factors, linearized series)
- Parameters (specification information)
- Diagnostics (information on the quality of the fit)

The graphical interface allows you to view these outputs and export them using drop-down menus .

The *Cruncher* avoids menu navigation by offering an identical automated export. The parameters and diagnostics are grouped into a single data file where the identifier is the name of the series. The exported file is called demetra_m.CVS, where "m" stands for matrix.

If the estimation is done in R, for example with {rjd3x13}, a list of lists groups the series, diagnostics, and parameters.

```
library("rjd3x13")
mod <- x13(my_series)
mod$result$final
```

Specifications

A specification is the set of parameters and constraints specific to X-13 or Tramo-Seats that are used to seasonally adjust a time series. Each algorithm has a set of default specifications that the user can customize. Thus, a specification will be attached to each series, stored in the *workspace* or in the R object.

More specifically, version 3 of JDemetra+ allows you to work with three versions of the same initial specification: `Reference Specification`, `Estimation Specification`, and `Result Specification`. The distinction between these three stages becomes useful when applying an estimation or refresh policy using the GUI or the *Cruncher* during annual or sub-annual campaigns.

- `Estimation Specification`: The set of constraints used by the algorithm to perform the calculations and obtain the expected series and indicators. If the user has not modified any parameters, it will be equal to one of the default specifications.

- `Result Specification`: Complete result of the estimation (Reg-Arima model coefficients, filter lengths and weights, etc.) used to derive the estimated series from the raw series. All parameters are determined here; there is no further identification or automatic search for effects. This specification can be applied to a (compatible) series.
- `Reference Specification`: Initially corresponds to the default specification selected by the user. During a refresh policy, it overrides the parameters of the `Estimation Specification`—even if those were specified by the user—where constraints are relaxed. If you want a parameter from the `Estimation Specification` to be preserved even though the refresh policy might re-identify or re-estimate it, it must be included in the `Reference Specification`.

The display and user modifications of these three types of specifications, as well as an example use case, are presented in Appendices .

Definition of specifications and the behavior of refresh policies differ from JDemetra+ version 2; details are provided in Appendices .

Workspaces

A *workspace* is a data structure specific to JDemetra+, consisting of a set of XML files organized into folders (image and link to doc) containing, among other things:

- External variables (calendar regressors, etc.)
- Custom calendars for the *workspace*
- Specifications (seasonal adjustment models)
- Multi-documents, also known as SA-Processing

At the first level, a *workspace* is the combination of a directory (containing XML files) and a master XML file that defines the structure of the folder and its contents. Both have the same name and must be located together in the same directory.

A *workspace* contains SA-Processing, the equivalent of subdirectories specific to JDemetra+, where SA-items are created. These are sub-structures containing all the information relating to a series (raw data, links to it, and the three types of specifications seen in the previous section). The *workspace* does not store results, but rather all the parameters needed to regenerate them. We complete this description in appendices .

The following section details the steps involved in building a production process by combining the tools described above.

Part 2: Building a production process

Two approaches

JDemetra+ allows to use of seasonal adjustment algorithms via R packages or the graphical user interface (GUI). The use of the GUI is generally motivated by a subsequent manual fine-tuning step, which is greatly facilitated by an organized view of the results and diagnostics, as well as the ability to modify specifications in a guided manner. However, using the GUI does not require the user to perform all operations manually with the mouse. There are two ways to automate the process: the *Cruncher* can be used to run estimations and export outputs, while the rjd3production and rjd3workspace packages can be used to bulk operations such as customizing specifications.

Producers who make little or no use of manual fine-tuning are increasingly inclined to use R functions directly and therefore only manipulate TS (Time Series) objects, freeing themselves from the *workspace* structure. However, these two approaches are not mutually exclusive; it is entirely possible to recreate a functional *workspace* (readable by the GUI and crunchable) from R objects.

In the rest of this section, we will first look at the case of using the GUI, then discuss the case of using R packages exclusively.

Using the graphical user interface

Using the GUI therefore requires organization into *workspace*(s). These can be created manually or via an R script, based on the following code

```
library("rjd3workspace")

# Creation of a workspace and SA-Processing
my_ws <- jws_new()
jsap1 <- jws_sap_new(my_ws, "sap1")

# Filling in with sa-item
```

```
y <- rjd3toolkit::ABS$X0.2.09.10.M # raw series
add_sa_item(jsap1, name = "serie_1", x = y, rjd3x13::x13_spec())

# Saving the workspace as an .xml file readable with the GUI
save_workspace(my_ws, "C:/my_folder/workspace_test.xml")
```

Selection and assignment of a set of calendar regressors

As a preliminary step to estimation, the selection of calendar regressors consists of determining which set of provides the best correction for each series. To do this, we suggest following these steps:

- Create a set of regressors to test, as detailed in Attal-Toubert (2012) and illustrated in appendices
- run automatic seasonal adjustment (possibly keeping a relevant predefined set of outliers, for example those corresponding to the COVID period) for all the pre-selected sets of regressors
- note the relative effectiveness of each set by penalizing residual trading days effects
- choose the set with the best score

A function in the {rjd3production} package covers these steps and returns a table where to each input series corresponds the best set.

```
my_context <- create_insee_context()
my_context$variables <- my_context$variables[c("REG1", "REG1_LY", "REG6", "REG6_LY")]
td_table <- select_regs(my_series, context = my_context)
```

- assign the selected set to each series of the *workspace* by declaring it in the corresponding specification

```
assign_td(td = td_table, jws = my_ws)
```

Customization of specifications

Beyond the choice of a set of calendar regressors, each series is likely to have specific characteristics that the producer wishes to impose: pre-specified outliers, a decomposition scheme, or an estimation period are the most common examples.

This customization can be done via the graphical interface for a small number of series or may require the execution of a script in the case of a large number of series.

In the following example, we modify one SA-Item (`jsai`):

```
library(rjd3workspace)

# reading the *workspace*
jws <- jws_open("C:/my_folder/workspace_test.xml")

# retrieving the sa-item in the *workspace* (here the first of the first SA-processing)
jsap <- jws_sap(jws, 1L)
jsai <- jsap_sai(jsap, 1L)

spec0 <- get_reference_specification(jsai)

spec1_with_outliers <- rjd3toolkit::add_outlier(
    x = spec0,
    type = c("AO", "AO", "LS"),
    date = c("2020-02-01", "2020-03-01", "2014-08-01")
)

spec2_with_new_span <- rjd3toolkit::set_basic(spec1_with_outliers, type = "From", d0 = "2012-01-01")

spec3_with_td <- rjd3toolkit::set_tradingdays(
    spec2_with_new_span,
    option = "UserDefined",
    uservariable = c("REG1.group_1", "REG1.group_2"),
    test = "None"
)

set_reference_specification(jsap, 1L, spec3_with_td)
```

Here we assign pre-specified sets of outliers to an entire *workspace* :

```
assign_outliers(outliers = outliers_table, jws = my_ws)
```

Outliers can be stored in an "*.yaml" file or imported from en existing *workspace*.

It is also possible to retrieve X13-Arima or Tramo-Seats specifications from X13-Arima-Seats software and make them compatible with JDemetra+. We describe this migration procedure in appendices .

Estimation

Once the specifications have been set in their initial version, before any modification based on the diagnostics, the estimation can be launched manually in the graphical interface. It is very common to automate this step with the *Cruncher*, which will directly export the output (series and diagnostics). This avoids opening the GUI by running the code below:

```
library("rjwsacruncher")
cruncher_and_param(
    workspace = "C:/my_folder/my_ws.xml",
    rename_multi_documents = FALSE,
    policy = "complete"
)
```

A subdirectory /output is created in the *workspace* directory with the files containing the series and the file containing the diagnostics and parameters (demetra_m.csv). The objects to be exported are selected by setting options in R. The corresponding code can be viewed here .

In the example above, the policy is "complete" (or "concurrent"), which is the only choice possible for an initial estimation. We will return to this parameter and the various refresh policies available in the third part in more detail.

Although it is possible to organize series into several SAProcessings in a JDemetra+ *workspace*, it should be noted that the *Cruncher* will always act on an entire *workspace*.

The path to the physical location of the data must be specified. This link is entered in the *workspace* when the data is assigned to it. If the data has been moved before the estimation is updated, the path must be updated. This can be done with the following functions in the case of an Excel workbook:

```
# Loading the workspace
my_ws <- jws_open("C:/my_folder/workspace_test.xml")

# Updating the path for the first SA-Processing
spreadsheet_update_path(
    jws = my_ws,
    new_path = "C:/my_data/My_Series.xlsx",
    idx_sap = 1
)
# Saving the workspace (wrinting the file on the disk)
```

```
save_workspace(my_ws, "C:/my_folder/workspace_test.xml")
```

The case of a ".txt" or ".csv" file is illustrated here .

Quality Report

A quality report is constructed by assigning each series a score based on diagnostics selected from the JDemetra+ output. This makes it possible to quickly identify the series with the poorest statistical quality and thus target manual fine-tuning. The statistical score can also be weighted by the importance of the series in the dissemination process to achieve an even more pragmatic selection.

The {JDCruncheR} package produces this report based on the diagnostics and parameters in the "demetra_m.csv" file generated by the rjwsacruncher::cruncher_and_param function. This file can also be generated directly via the GUI using drop-down menus .

```
BQ <- JDCruncheR::extract_QR("C:/Workspaces/my_ws/Output/SAProcessing-1/demetra_m.csv")
```

The extraction parameters are detailed in the documentation for the `extractQR` function.

A set of diagnostics, detailed in the appendices , relating in particular to residual seasonality, residual trading day effects, and the whiteness of the residuals, are selected. Each of these is assigned a rating ("Good," "Uncertain," "Bad," or "Severe") according to thresholds that can be viewed using the option below:

```
library("JDCruncheR")
getOption("jdc_thresholds")
```

These thresholds can be configured using the `JDCruncher::set_threshold()` function.

Each modality corresponds to a grade, so ("Good," "Uncertain," "Bad," or "Severe") will become (0, 1, 3, 5). The score for a series is the weighted average of its scores, and its quality is lower when the score is higher. The residual seasonality tests on the seasonally adjusted series and on the irregular have the highest weights. The complete score equation is detailed in the appendices .

This equation can also be configured with the `score_pond` argument of the compute_score() function.

The result can be exported to Excel, where it is organized as follows by default:

- a sheet with the diagnostic methods for each series
- a sheet with the values, including the score, the weighted score, and the main contributions to the score

The user can enrich this file and document the manual fine-tuning of the *workspace*. Sorting by descending (weighted) score yields the expected selective editing.

Below is an excerpt from the Excel export of the quality report, by default:

series	score	f_residual_td_on_i	residuals_kurtosis	residuals_normality	residuals_independenc
X0.2.06.10.M	60	Good	Bad	Good	Bad
X0.2.11.10.M	250	Severe	Good	Bad	Good
X0.2.10.10.M	250	Severe	Good	Good	Good
X0.2.20.10.M	545	Severe	Uncertain	Bad	Uncertain

series	score	f_residual_td_on_i	residuals_kurtosis	residuals_normality	residuals_indepe
X0.2.06.10.M	60	0,0815	0	0,6324	0
X0.2.11.10.M	250	0	0,6252	0	0,6774
X0.2.10.10.M	250	0,0006	0,8809	0,1105	0,7377
X0.2.20.10.M	545	0	0,082	0	0,0167

Guided by the quality report, the producer modifies the specifications to improve seasonal adjustment, generally using the graphical interface. To generate the final output, particularly the series, they can restart the *Cruncher*, no longer with the policy="complete" option but using a refresh that re-estimates only the coefficients of the pre-adjustment model ("policy=regarimaparmeters") and retains all of the user's modifications. We provide more details on this point in the appendices .

```
library("rjwsacruncher")
cruncher_and_param(
    workspace = "C:/my_folder/my_ws.xml",
    rename_multi_documents = FALSE,
    policy = "regarimaparameters"
)
```

Using only R packages

All of the steps described above can be performed without using a *workspace* structure and remaining in the R environment.

Estimation

In this case, we use the functions `rjd3x13::x13()` (or `rjd3tramoseats::tramoseats()`) to perform the estimations. Estimation functions work on one TS object which will require to set up loops or "lapply" type functions.

```
# for one series
mod <- x13(my_series, my_spec)
str(mod)
```

Diagnostics and specifications

In R, you can access the same diagnostics as from the GUI and the *Cruncher* by navigating through the object produced by the estimation (list of lists), which you can directly organize into a customized quality report. To take advantage of the existing formatting, simply rewrite a crunchable *workspace* and regenerate the same report as in the GUI approach. To remain in one environment, manual modification of the specifications based on the diagnostics will in this case be done directly in the R script, as in the example below:

```
spec0 <- rjd3x13::x13_spec()

spec1_with_outliers <- rjd3toolkit::add_outlier(
    x = spec0,
    type = c("AO", "AO", "LS"),
    date = c("2020-02-01", "2020-03-01", "2014-08-01")
)
```

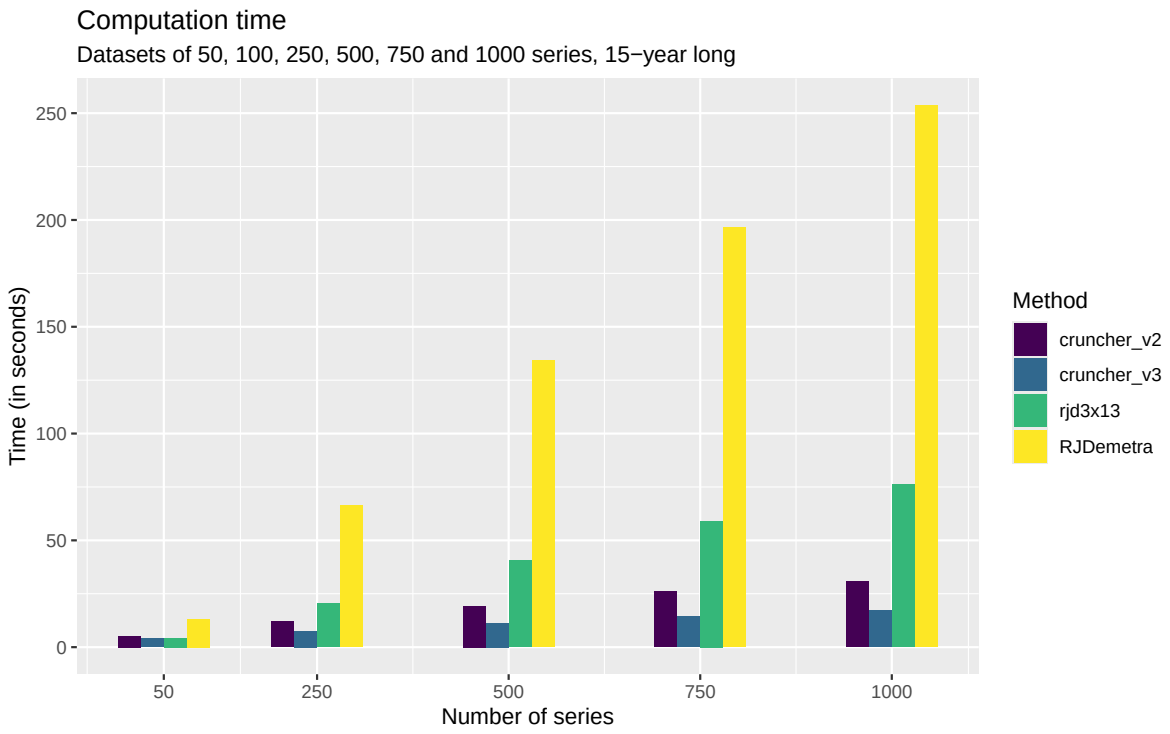
The loop of reading diagnostics and modifying parameters is less fluid, which encourages the use of the graphical interface approach in cases requiring intensive manual fine-tuning. To update specifications in R based on diagnostics, scripts must be modified directly, which is slower and more prone to error for analysts, especially those unfamiliar with R. The R-only approach makes storing and versioning specifications more complex.

The diagnostics used to construct the score presented above can be obtained directly in R, which allows the selective editing procedure to be replicated. The code for obtaining this output can be found in the appendices.

The R approach also eliminates any portability issues, even though updating the link between a *workspace* and the data is now simple and reliable.

Performance

We compared the performance of the two approaches. The graph below shows the computation times for seasonally adjusting a dataset, with outlier detection and calendar effect estimation with a preselected set of regressors, using the R packages (v2 and v3) or the cruncher (v2 or v3):



For a series number close to one thousand, the estimation with `rjdx13` takes less than three minutes, which is still very reasonable for most users but far behind the *Cruncher*, which completes the same process in about twenty seconds.

Once the production process is in place, it is recommended to validate or adjust the settings annually and to adopt an infra-annual production strategy, generally monthly or quarterly, allowing the seasonally adjusted data to be updated when new raw data becomes available. This issue is tackled in the next part.

Part 3: Infra-annual and annual campaigns

Infra-annual campaigns

The objective of an infra-annual campaign is to quickly produce an updated SA series when a new raw data point is available and the recent past of the raw series may be revised. The seasonally adjusted series has two sources of revision: the raw data and changes to the final seasonal factor estimation models. The seasonal adjustment guidelines recommend minimizing these so-called technical revisions (see Chapter 6). A common method for achieving this objective is to use the predicted seasonal factors, including the predicted calendar effect, calculated during the last annual campaign. However, JDemetra+ also offers a continuum of refresh methods that allow to gradually loosen the constraints on the pre-adjustment and control the revisions of the linearized series before the decomposition step, which will be completely redone. All of these refresh policies are presented in the documentation , among which the "lastoutliers" option is particularly recommended. In this case, we allow ourselves to re-identify the outliers over the last year. Manual fine-tuning in the infra-annual context will focus on revisions and therefore outliers, avoiding too many parameter changes. This refresh is possible from the graphical interface, with the *Cruncher* by specifying the "policy" argument of the `rjwsacruncher::cruncher_and_param()` function but also directly in R with the functions `rjd3x13::x13_refresh()` or `rjd3tramoseats::tramoseats_refresh()`.

For infra-annual estimation with the *Cruncher*, the following R code can be used:

```
library("rjwsacruncher")
cruncher_and_param(
    workspace = "C:/my_folder/my_ws.xml",
    rename_multi_documents = FALSE,
    policy = "lastoutliers"
)
```

Table 4: Implementation of a refresh policy

Data type	Tools
Workspace	- GUI (Drop-down menus)

Data type	Tools
TS objects in R	- `rjwsacruncher::cruncher_and_param( ... , policy=,)` - `rjd3workspace::jws_refresh()` - `rjd3x13::x13_refresh()` - `rjd3tramoseats::tramoseats_refresh()`

The use of forecast seasonal coefficients (S_f) simply requires exporting them at the end of the installation process, then at the end of each annual campaign, in order to perform the calculation: $Y_{sa} = Y_{raw} - S_f$ or $Y_{sa} = Y_{raw}/S_f$, in the case of a multiplicative model. To potentially take into account a revision of the recent past of the raw series, exporting past coefficients is also useful. The series of forecast coefficients can be generated by the *Cruncher* by specifying it in the options, as shown below

```
options(default_tsmatrix_series = c("sa", "sa_f", "s", "s_f"))
```

or in R by defining a user-defined output as shown below

```
mod <- x13(y, userdefined = c("s", "s_f"))
str(mod$user_defined)
```

Annual campaigns

An annual campaign consists of updating seasonal coefficient estimation models to verify that the parameters used do indeed produce seasonally adjusted series with the required properties (absence of residual seasonality, absence of residual calendar effects, etc.).

This update can be done by comparing the current or reference models used up to the time of the campaign with an automatic estimate. In most cases, the automatic estimation will be constrained by past parameters such as the selection of calendar regressors or certain pre-specified outliers, particularly those from the COVID period. For ease of comparison, it may be useful to calculate the quality report scores described in Part 2 for the “reference” or “current” *workspace* and the “automatic” *workspace*. A lower score for the automatic estimate means that this is preferable, and a new *workspace* (“working”) can then be created by merging the “current” and “automatic” *workspaces*, as detailed in the appendices .

This approach requires working on comparable raw series. If these have undergone significant revisions (re-basing, change of source, etc.), we return to an approach similar to the installation of a new process.

Main steps:

- Update the reference *workspace* ("ref") using the new raw data and statistical parameters from the previous annual campaign, possibly modified by the same strategy as a sub-annual campaign (e.g., "lastoutliers").
- Automatic estimation ("auto") on the new raw data, probably maintaining certain parameters (set of calendar regressors, some or all of the outliers "pre-specified" by the user, etc.)
- calculation of scores for the two *workspaces* "ref" and "auto"
- merging of the "auto" and "ref" *workspaces* into a new 'working' *workspace* according to the score value of each series
- editing of a quality report on the "working" *workspace*
- Manual review with selective editing, as in the installation phase
- Generation of the final output with the *Cruncher*, as in the installation phase

If forecast coefficients are used during infra-annual campaigns, the *workspace* from the previous annual campaign can be updated with a policy "last outliers" to return to the case described above.

Details on how the data is organized and code snippets are provided in the vignette {rjd3production} package .

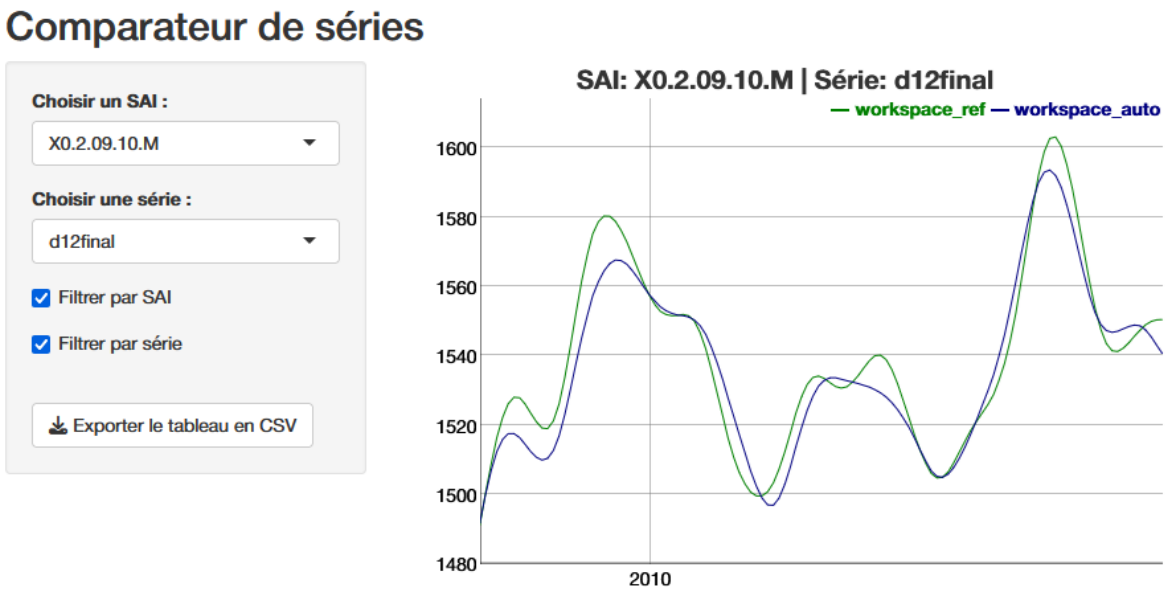
Numerical impact of a change in parameters

In light of the (bad) diagnostics, identified in particular through the quality report, the producer can intervene to modify the parameters and improve the statistical quality of the adjustment. For example, they can modify a set of calendar regressors and eliminate the residual calendar effects that they had previously observed. Beyond improving statistical test results, it is often useful to visualize the numerical impact of a parameter change or re-estimation, which allows revisions to be quantified and visualized, but also certain settings to be validated, particularly the choice of outliers based on "expert knowledge." Exporting the series produced is one solution, but if you want to make these comparisons before you have finished working on a *workspace*, you can use the `compare()` function from the rjd3production package, which avoids generating intermediate outputs. This

allows you to read the series directly in two or more *workspaces*, each corresponding to a version of the parameters or a state of the estimate, as shown in the following code:

```
df <- compare(path_ws_ref, path_ws_work)
run_app(df)
```

The function launches a shiny application that provides a graphical comparison, below is the final trend (d12), and the ability to download the series values to analyse the revisions.



Conclusion

Ensuring the statistical quality of seasonally adjusted series requires the implementation of a production process that, in light of diagnostics, allows for easy and large-scale customization of estimation parameters. The use of the open-source software JDemetra+ and its R tools makes it possible to achieve this goal by offering various automation options while providing access to detailed manual fine tuning. We have seen that the decision to integrate the graphical user interface into a production process is fundamental to the organization of data. The main reasons for this should be the use of manual fine-tuning and performance. However, an organization based solely on R does not exclude ad hoc use of the GUI or *Cruncher*. The organization of operations and the use of tools as described in

this article are, of course, only suggestions, which we hope will provide practical answers to the many producers of SA series who wish to use JDemetra+.

In order to communicate with their users, they often need reporting tools that break down the effects of seasonal adjustment on changes in a given series, in particular to highlight the various contributions to revisions. Such tools are often *in-house*, but there are packages or plug-ins linked to JDemetra+, often still in version 2, that allow to display this type of information. It would be interesting to follow up on this article by offering a review of open-source reporting tools and making available an R package that allows users to take full advantage of version 3 of JDemetra+.

Appendices

Calendar effect correction

There are two possible procedures for generating calendar regressors: creating customized regressors as shown below, or using predefined regressors in JDemetra+, which also requires defining a reference calendar. Version 3.x of the GUI allows you to automatically [select)(https://doc.jdemetra.org/a-sa-pretreatment#calendar-correction) from the predefined regressor sets, based on statistical tests.

Instead of a simple national calendar , JDemetra+ allows you to create "chained" or "composite" with the GUI or rjd3toolkit .

Creating a reference calendar

To create a French calendar based on French public holidays, you can use the `create_french_calendar()` function from the {rjd3production} package:

```
cal_FR <- create_french_calendar()
```

This function can be adapted to any national calendar.

Generation of regressor sets

Using the French calendar, it is possible to generate calendar regressors based, for example, on the usual groups used by INSEE:

```
regs <- create_insee_regressors(start = c(2000, 1), frequency = 12, length = 360)
sets <- create_insee_regressors_sets(start = c(2000, 1), frequency = 12, length = 240)
names(sets)
```

```
[1] "REG1"    "REG2"    "REG3"    "REG5"    "REG6"    "LY"      "REG1_LY"
[8] "REG2_LY" "REG3_LY" "REG5_LY" "REG6_LY"
```

Quality Report: Diagnostics and R code

Diagnostics selected by default when using X-13-Arima[3] are as follows:

Table A1: Diagnostics

Diagnostic	Variable name in demetra_m.csv file (Cruncher or GUI)	Access with rjd3x13
Skewness test on Reg-Arima residuals	residuals.skewness:3	m_sa$result$preprocessing$diagnostics$skewness
Ljung-Box test on Reg-Arima residuals	residuals.lb2:3	m_sa$result$preprocessing$diagnostics$lb2
Normality (Doornik-Hansen) test on Reg-Arima residuals	residuals.doornikhansen:3	m_sa$result$preprocessing$diagnostics$doornikhansen
QS-Test for residual seasonality on final sa series	diagnostics.seas-sa-qs:2	m_sa$result$diagnostics$seas.qstest.sa
QS-Test for residual seasonality on final irregular	diagnostics.seas-i-qs:2	m_sa$result$diagnostics$seas.qstest.i
F-Test for residual seasonality on final sa series	diagnostics.seas-sa-f:2	m_sa$result$diagnostics$seas.ftest.sa
F-Test for residual seasonality on final irregular	diagnostics.seas-i-f:2	m_sa$result$diagnostics$seas.ftest.i
F-Test for residual trading days effect on final sa series	diagnostics.td-sa-last:2	m_sa$result$diagnostics$td.ftest.sa
F-Test for residual trading days effect on final irregular	diagnostics.td-i-last:2	m_sa$result$diagnostics$td.ftest.i
Mean equality test in and out of sample	diagnostics.fcast-outsample-mean:2	m_sa$user_defined$`diagnostics.fcast-outsample-mean`
Variance equality test in and out of sample	diagnostics.fcast-outsample-variance:2	m_sa$user_defined$`diagnostics.fcast-outsample-variance`
X-11 decomposition: Q-M2 statistic	m-statistics.q-m2	m_sa$result$mstats$qm2
X-11 decomposition: M7 statistic	m-statistics.m7	m_sa$result$mstats$m7

In R, the diagnostics are extracted from the “m_sa” object containing all the estimation results and obtained with the code below.

```
m_sa <- x13(my_series, my_spec, userdefined = c("diagnostics.fcast-outsample-mean","diagnostics.fcast-out
str(m_sa)
```

More information on the tests is available in the documentation

[3]When using Tramo-Seats, this also works, but in this version, the diagnostics specific to X-13-Arima are simply removed and not replaced

Score equation

$$
\begin{aligned}
score_total = & 30 * grade_qs_residual_s_on_sa \\
& + 30 * grade_f_residual_s_on_sa \\
& + 30 * grade_f_residual_td_on_sa \\
& + 20 * grade_f_residual_td_on_i \\
& + 20 * grade_qs_residual_sa_on_i \\
& + 20 * grade_f_residual_sa_on_i \\
& + 15 * grade_oos_mean \\
& + 10 * grade_oos_mse \\
& + 15 * grade_residuals_independency \\
& + 5 * grade_residuals_skewness \\
& + 5 * grade_residuals_homoskedasticity \\
& + 5 * grade_q_m2 \\
& + 5 * grade_m7
\end{aligned}
$$

Annual campaign: comparison of workspaces

Creation of WS_work

The **WS_work** is a merger between **WS_ref** and **WS_auto**. A score is calculated for both *workspaces* and, for each series, the SA-Item is retrieved from the *workspace* with the lowest score between WS_auto and WS_ref.

In an annual campaign process, we will use the `transfer_sa_item()` function from {rjd3workspace} to merge **WS_ref** and **WS_auto**.

```
# save WS_ref as initial version of WS_work
WS_auto <- load_workspace(file = "./WS/WS_auto.xml")
WS_work <- load_workspace(file = "./WS/WS_work.xml")

sap1_auto <- .jws_sap(jws = WS_auto, idx = 1)
sap1_work <- .jws_sap(jws = WS_work, idx = 1)

# update ws_work with sa-items from ws auto when necessary
transfer_series(
    jsap_from = sap1_auto,
```